

C++ Contracts and Coroutines

Document number	D3251R0
Date	2024-04-23
Reply-to	Peter Bindels <dascandy@gmail.com>
Targeted subgroups	SG21

§ 1 Introduction

The current papers on contracts [\[1\]](#), [\[2\]](#) discuss in detail how the contracts facility inside C++ should work, with the perspective of having a future C++ standard version, and how a code base composed solely of C++26+ code using contracts will work. In this paper, we'll take a look into how contracts would interact with coroutines.

The paper is not required for the MVP; it defines behavior for a currently ill-formed construct. It does have very little impact on the contents as it mostly provides a rationale and approach for the reason not to forbid coroutines to have contracts, and as such it could be considered for the C++26 deadline. The paper takes into account the discussion documented at [\[3\]](#).

This paper is beside [\[4\]](#). The conclusions from both papers and the standard / P2900 impact of both is identical, but this paper goes into detail on how contracts can work on the return channel of a coroutine, showing why that does not need a future paper or future changes.

§ 2 Detail

§ 2.1 Contracts on the coroutine function interface

Within the C++26 contracts proposal, we have the ability to specify a `pre` and a `post` condition on a function declaration. These `pre` and `post` conditions indicate the state as entering the function, typically asserting properties about the parameters but potentially about other parts of the program state, as well as the state and return value of a function when returning from it.

A coroutine is a function that in its implementation contains one or more suspension points. The function may be suspended at each of these points, with control being passed back to the calling function early and leaving the function state in a resumable state. In doing so it can yield a value to the calling function,

or just suspend while awaiting something else. A coroutine will at some point typically end and signal that it has no further computation to do, while optionally returning a final computed value.

The definition we currently have for contracts on function prototypes does not distinguish between coroutine function declarations and non-coroutine function declarations. As whether a function is a coroutine is not visible on the interface definition, this would be impossible without transferring the knowledge of whether something is a coroutine to the declaration, modifying the basic design of coroutines. This limits the kinds of contracts we could place on coroutines to only those that assert on the function's entry state, and those that assert on the coroutine's suspended state.

In a way, the restriction on contracts not being available on coroutine functions is academic; any function `F()` that is a coroutine where one would like to apply contracts can be indirected through a single other function `auto G() { return F(); }`, where `G` is not a coroutine and is eligible for any contracts the caller would like to use. This is also the behavior that users will expect from such contracts - if you were to indirect through a function `G` to get around this limitation, the contracts applied to `G` will necessarily do exactly what they would be doing on `F`. There is a small asterisk to address, which is that the postcondition in a coroutine cannot refer to any `const` arguments as even those may have been moved-from, where the postcondition on an equivalent non-coroutine could be stated on those arguments. It is worth a consideration whether or not the benefit of being able to state a postcondition on the awaitable return object of a coroutine outweighs the downsides of having a postcondition that most of the time, but not always, can refer to arguments. I would not see a major problem in rejecting a coroutine implementation for a function that contains postconditions to avoid this corner case.

§ 2.2 Contracts on the logical interface of a coroutine

A coroutine has an interface that goes beyond what the returned state is at the point of first suspension. The function might not even have started executing at that point, leaving very little to be asserted. The logical interface of a coroutine however, offers the ability to yield multiple values, as well as return a value at its end. It would be desirable for the coroutine author to be able to indicate postconditions on the values being yielded and/or returned in some way, which would need to be carried by the interface of the function without making it somehow annotated as being a coroutine.

The solution we propose is to have the coroutine's awaitable and promise type bear the responsibility of carrying the postcondition contract for the values being returned. It is a general solution that does not require the function to be a

coroutine, but instead specifies what the yielded and/or returned value are to be constrained to.

The coroutine's returned type plus its associated promise type have the concrete functions that would, in a theoretical setup, be the ones where the contract would naturally be applied. The promise type's `yield_value` and `return_value` functions receive the yielded or returned values and are in an ideal position to bear an assertion as a precondition over them, where the future type's retrieval functions (such as `await_resume`) carry the same assertion as postcondition. These places are already eligible to receive a precondition and/or postcondition, and the ability to do this requires zero change in the current Contracts paper.

In effect, this places a precondition/postcondition pair on the awaitable as functioning as a channel for the value it carries, with the contract on one hand being asserted on the generating side, and on the other side being made available on the receiving side. This both ensures that if the contract is violated we are made aware at the generating side, and that the contract being valid is available to the compiler on the receiving side for optimization (if applicable).

The way to potentially implement this is to make the assertion to be used on the yielded or returned value(s) a part of the returned type - for example, by making it a custom returned type with the contracts asserted, or as template parameters used in a general type to represent the contracts that is carried into the pre- and postcondition. That said, to implement this requires no changes to the current specification of [2].

The part that does require modification is removing the ban on contracts existing on a function that is a coroutine. As it stands, this forbids the precondition from existing, and does not allow assertions to be made on the awaitable and promise object that is returned from the function. However the latter may seem useless, the former is definitely useful and currently forbidden.

§ 2.3 Coroutine invariants

Coroutines may have invariants that hold during the execution of the coroutine. As a simple example, a generator could assert that the values generated are strictly increasing. These are, similar to the above concept of returned and yielded values having a postcondition that is passed into the return type as a template parameter, a contract that is applied by the `promise_type` on the suspend and resume points. It can similarly be passed in as a template parameter, hardcoded in some way, or in any other way made available to the promise type to assert.

§ 3 Impact

Change to [2] to no longer disallow coroutines to have contracts.

For a coroutine to have ~~precondition or~~ postcondition specifiers is ill-formed. ~~Support for coroutines is expected to be proposed in a future extension (see Section 2.3).~~ Precondition specifiers on a coroutine work identical to regular functions.

~~This requirement is enforced on the function definition since whether a function is a coroutine cannot be known until a use of `co_return`, `co_await`, or `co_yield` is found enclosed by the function body.~~

Pre- and postconditions that apply to the values returned from within the coroutine should be specified as pre- and postconditions on the promise type.

Rationale to be added to P2899 is found above.

§ 4 Open questions

Q: The reasoning listed above applies to a subset of all uses of coroutines. It includes generators, channels, future/promise and similar kinds of uses of coroutines. The author of the paper does not have a full enough grasp of coroutines to know for sure that this design satisfies all types of uses of coroutines, and invites feedback from anybody that can help confirm, deny, or understand the problem space further.

§ 5 Acknowledgements

Thanks to Ville Voutilainen for the fruitful discussion on Mattermost leading to this paper. Thanks to Timur Doumler for raising the idea of coroutine invariants. Thanks to Ran Regev for assistance with acquiring the information needed to write this paper. Thanks to Andrzej Krzemiński for writing [5]; its conclusion with regards to coroutine preconditions is identical to this paper, and if it were not for the explanation on contracts on coroutine return channels this paper would have nothing to add. And a major thanks to all of SG21 for the work done on contracts so far; this paper would not be possible without a very well formed P2900 to write it against.

Thanks to Dawid Pilarski and Tom Honermann for reviewing this paper before publishing.

§ 6 References

1. [P2899 \(http://wg21.link/p2899\)](http://wg21.link/p2899).
2. [P2900 \(http://wg21.link/p2900\)](http://wg21.link/p2900).
3. [Kona 2023 meeting on coroutines with contracts \(https://wiki.edg.com/bin/view/Wg21kona2023/SG21MeetingMinutes2023-11-10\)](https://wiki.edg.com/bin/view/Wg21kona2023/SG21MeetingMinutes2023-11-10).

4. [P2957 \(http://wg21.link/p2957\)](http://wg21.link/p2957).
5. [\(P2957\)](#).